AI-Powered LinkedIn Content Agent

High-Level Backend Architecture

A SaaS platform that helps professionals maintain a consistent LinkedIn presence through AI-assisted ideation, human approval, scheduling and reliable publication.

Architecture brief | MVP-first | LinkedIn-first

1. Product vision and MVP boundary

The product is an AI-assisted content operations system for professionals who want to publish consistently on LinkedIn without manually starting from a blank page every few days. The product optimizes for consistency, relevance and user control - not guaranteed virality.

The user problem

Professionals struggle to sustain a relevant posting cadence while job searching, building authority or growing a business.

The value loop

Configure strategy -> generate options -> review -> approve -> schedule -> publish -> learn.

The trust model

AI drafts and recommends. The user remains the final editor and explicitly approves every publication.

MVP capabilities

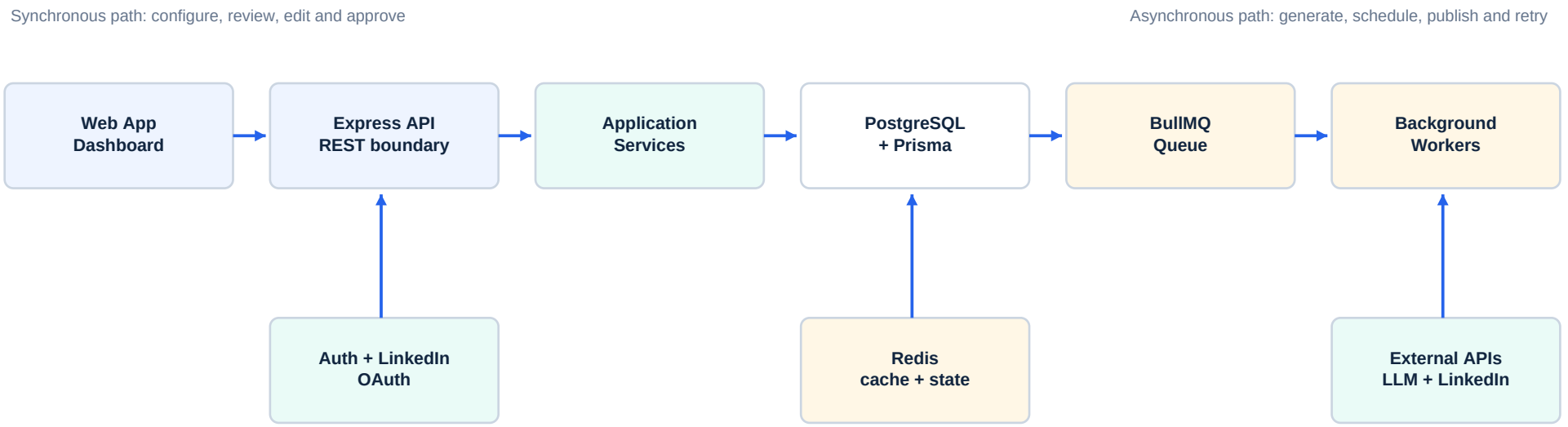
- Account registration and sign-in.
- Authorized connection to one LinkedIn account through OAuth.
- A reusable content strategy: profession, audience, goals, tone, cadence, preferred topics and exclusions.
- Generation of multiple LinkedIn post options with an optional image suggestion or generated image.
- Review, editing, approval and rejection workflow.
- Scheduling and background publication of approved posts.
- Publication status, retry visibility and a minimal activity history.

Explicitly outside the MVP

- Instagram, YouTube and other publishing channels.
- Video generation and voice workflows.
- Automatic publication without approval.
- Advanced analytics, trend prediction and self-optimizing recommendations.
- Team workspaces, billing tiers and enterprise governance.

2. Architecture overview

A modular monolith is the recommended starting point: one codebase with clear business boundaries, one primary relational database and separately runnable API and worker processes. This keeps delivery simple while preserving a path to scale high-load components independently.



Key runtime separation

- The API handles interactive requests and returns quickly.
- BullMQ records asynchronous jobs in Redis; workers execute generation and publication work outside the request-response cycle.
- PostgreSQL is the system of record for users, strategy, drafts, approvals, schedules and publication outcomes.
- Redis is transient infrastructure for queues, rate limits, locks, idempotency markers and selective caching - never the sole source of business truth.

3. Major components

Frontend

Dashboard for onboarding, strategy configuration, draft review, editing, approval, scheduling and status monitoring.

API

Express REST boundary. Validates requests, authenticates users, applies authorization and delegates use cases.

Auth

Application sessions plus LinkedIn OAuth. Stores provider tokens encrypted and refreshes or reconnects as required.

AI service

Provider-neutral adapter for OpenAI, Groq or Claude. Builds versioned prompts, requests structured output, validates responses and records model metadata.

Scheduler

Turns approved future publications and recurring generation preferences into durable queue jobs.

Queue

BullMQ manages waiting, delayed, active, completed and failed jobs using Redis, including retry and backoff policies.

Workers

Separate processes for content generation and LinkedIn publication. Enforce idempotency and write outcomes to PostgreSQL.

Database

PostgreSQL stores durable product state. Prisma provides schema, migrations, typed access and transactions.

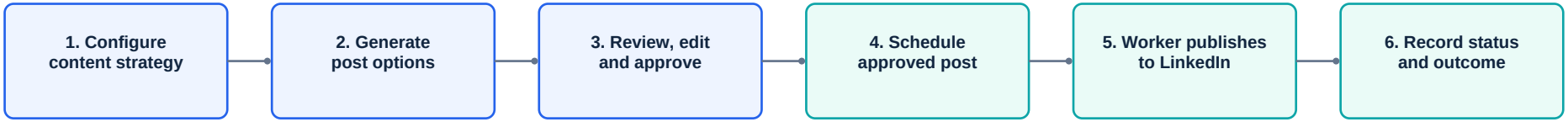
Cache and coordination

Redis supports BullMQ, short-lived cached reads, distributed rate limiting, locks and idempotency keys.

External APIs

LinkedIn OAuth/API for account authorization and publishing; an LLM provider for generation; optional image provider later.

4. Core data flow



Human approval is mandatory before publication in the MVP.

Generation flow

- A recurring scheduler or explicit user action creates a content-generation job.
- The worker loads the user's active strategy and recent drafts from PostgreSQL.
- The AI adapter sends a versioned prompt and requires a structured response.
- Validated options are persisted as drafts; the user is notified that review is available.

Publication flow

- Only an approved draft can be scheduled.
- A delayed BullMQ job contains identifiers, not full sensitive records.
- At execution time the worker reloads current state, verifies approval and checks an idempotency key.
- The worker calls LinkedIn, stores the external post identifier and marks the publication successful.
- Transient failures retry with bounded exponential backoff; permanent failures are recorded for user action.

Failure and consistency rules

- Never publish from an unapproved or stale draft.
- Use an idempotency key per scheduled publication to prevent duplicates.
- Database state is authoritative; queue jobs are delivery mechanisms.
- Use transactions for coupled state changes; keep failures visible and redact sensitive data from logs.

5. Proposed technology stack

Runtime and language	Node.js + TypeScript	Strong asynchronous I/O model, shared language across frontend/backend and strict typing.
HTTP application	Express	Small, well-understood API layer with explicit middleware and routing.
Primary database	PostgreSQL	Reliable relational storage for users, strategies, drafts, schedules and audit history.
Data access	Prisma	Typed queries, schema migrations and productive local development.
Queue and scheduler	BullMQ + Redis	Durable delayed jobs, retries, concurrency control and worker coordination.
Containers	Docker + Docker Compose	Reproducible local environment for API, worker, PostgreSQL and Redis.
AI providers	OpenAI, Groq or Claude through an adapter	Provider flexibility, structured generation and testable mocks.
Social integration	LinkedIn OAuth + API	Authorized account connection and publication to the user's LinkedIn presence.
Frontend	React-based web app	Focused dashboard for setup, approval, scheduling and operational status.
Quality	Zod, Vitest, Supertest, ESLint, Pino	Runtime validation, automated tests, static quality and structured logs.

6. Suggested module organization

Organize by business capability, with shared infrastructure kept behind small interfaces. The exact filenames can evolve; the boundaries matter more than the tree.

```
src/
app/ composition, middleware, routes
modules/
auth/ sessions, authorization, LinkedIn OAuth
users/ account and preferences
strategies/ audience, tone, topics, cadence
content/ ideas, drafts, revisions, approvals
scheduling/ schedules and recurring triggers
publishing/ LinkedIn publication lifecycle
ai/ prompts, schemas, provider abstraction
audit/ important user and system actions
infrastructure/
database/ Prisma client and persistence adapters
queue/ BullMQ queues, job contracts, policies
cache/ Redis connection and cache helpers
external/ LinkedIn and LLM provider clients
logging/ structured logs and request context
workers/ generation and publication processors
shared/ errors, validation, utilities, types
prisma/ schema and migrations
tests/ unit, integration and contract tests
```

Dependency direction

- Routes and controllers depend on application services.
- Application services depend on domain contracts, not vendor SDKs.
- Infrastructure implements database, queue and external-provider contracts.
- Workers call the same application services used by interactive flows.

Initial domain records

- User and Session
- LinkedInConnection
- ContentStrategy
- PostDraft and DraftVersion
- Approval
- PublicationSchedule
- PublicationAttempt
- PromptVersion and AIExecution
- AuditEvent

7. Delivery roadmap

MVP foundation	A secure product shell	Auth, strategy setup, database, API, dashboard foundation, tests	Payments, teams, advanced roles
MVP content loop	Useful drafts ready for review	Provider adapter, structured generation, draft versions, edit/approve/reject	Trend ingestion, automatic optimization
MVP publishing	Reliable scheduled LinkedIn posts	OAuth, delayed jobs, worker, retries, idempotency, status history	Multiple platforms, autonomous posting
Phase 2: feedback loop	Recommendations improve with evidence	LinkedIn metrics ingestion, normalized analytics, post comparisons, strategy suggestions	Full attribution and prediction
Phase 3: growth SaaS	Commercial and multi-channel product	Subscriptions, quotas, Instagram, teams, templates, richer media	YouTube automation and voice

Recommended MVP milestone order

- 1. Product skeleton and account authentication.
- 2. Strategy creation and editing.
- 3. Manual generation request and saved draft options.
- 4. Review, revision and explicit approval.
- 5. LinkedIn OAuth connection.
- 6. Schedule creation, BullMQ delayed job and publication worker.
- 7. Retry handling, idempotency, audit history and operational hardening.

8. Architecture principles

MVP before platform

Implement the shortest complete user journey before analytics, multiple networks or advanced agent behavior.

Modular monolith first

Preserve clear module boundaries without accepting the operational cost of microservices prematurely.

Idempotency over hope

A retry must never create a duplicate LinkedIn post or duplicate durable record.

Provider independence

LLM and LinkedIn SDK details stay behind adapters to support testing and future change.

Observable by design

Use request/job correlation IDs, structured logs, job metrics and clear user-facing status.

Human control by default

Generation and recommendation may be automated; publication requires explicit approval in the MVP.

Async work is durable

Generation and publication must survive API restarts and expose retry and failure state.

Database as source of truth

Redis accelerates and coordinates; PostgreSQL owns durable product state.

Privacy and least privilege

Request minimum LinkedIn permissions, encrypt tokens, redact logs and define retention rules.

Test behavior at boundaries

Unit-test decisions; integration-test database and queue adapters; contract-test external clients.

Architectural decision summary

Begin with one repository and two deployable processes: an HTTP API and a worker. Use PostgreSQL for durable state, Redis/BullMQ for background work, and provider adapters for LinkedIn and the selected LLM. Split services only when measured scale, reliability or team ownership requires it.